

Implement Client Script and UI Policy (Incident)

	Name	NM.Id
Team Leader	Yamuna devi.A	asaluau020224128037
Team Members	Kalaiselvi.S	asaluau020224128018
	Keerthana.M	asaluau020224128020
	Kavitha.V	asaluau020224128019
	Mageshwari.R	asaluau020224128021

1. Objective

The objective of the Implement Client Script & UI Policy (Incident) project is to demonstrate how Service Now client-side controls can be used to enforce data integrity on Incident records. This implementation showcases how UI Policies and Client Scripts can dynamically make field's mandatory, auto-populate values, control field behavior, and prevent record submission when required conditions are not met. The solution ensures that Incidents are created with complete and valid information, improving data quality and consistency across the platform.

2. System Architecture

This flow chart shows how Client Script and UI Policy work together in a Service Now Incident form.

Flow:

- User opens or creates an Incident.
- Incident Form is displayed in the browser.
- Client Script validates fields and performs actions when the form changes.
- UI Policy controls field visibility, mandatory status, and read-only settings based on conditions.
- When the user submits the form, data is sent to the Service Now server.

- The server validates and stores the data in the Incident Database.
- Finally, the updated Incident record is displayed to the user

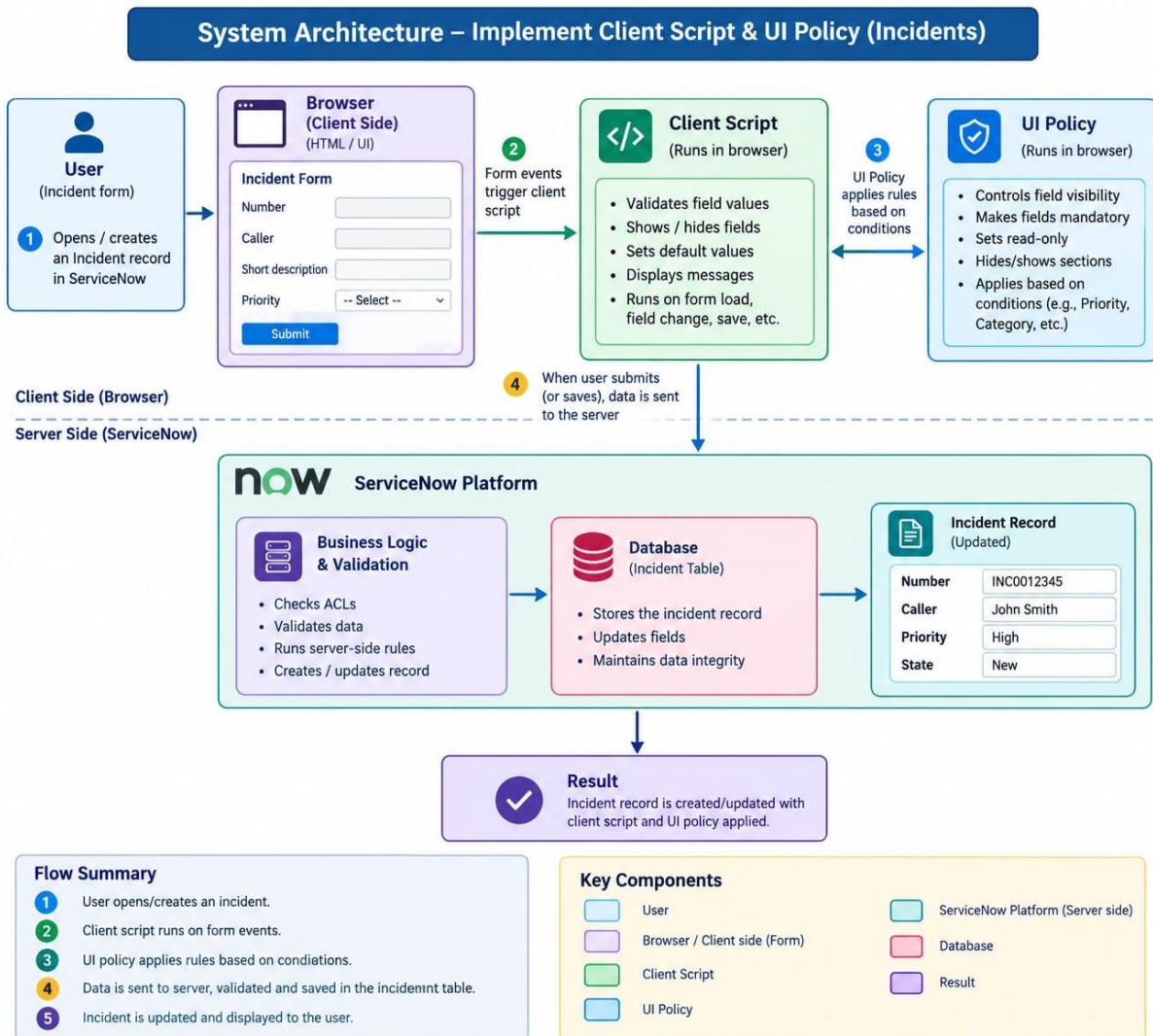


Figure1.System Architecture – Implement Client Script And UI Policy

4. System Requirement

4.1. Software Requirement

1. Service Now Platform – To create and manage Incident records.
2. Web Browser – Chrome, Edge, or Firefox to access Service Now.

3. Client Script – Used to validate fields, set values, and respond to user actions on the Incident form.
4. UI Policy – Used to make fields mandatory, visible/hidden, or read-only based on conditions.
5. Incident Management Module – To create, update, and manage incident records.
6. JavaScript Support – Required for writing and executing Client Scripts.
7. User/Admin Access – Appropriate Service Now roles and permissions to configure Client Scripts and UI Policies.

4.2. Hardware Requirement

1. Computer/Laptop – To access and configure Service Now.
2. Processor – Intel Core i3 or equivalent and above.
3. RAM – Minimum 4 GB (8 GB recommended).
4. Storage – Minimum 10 GB free space.
5. Display – Standard monitor/laptop screen.
6. Internet Connection – Stable broadband/Wi-Fi connection.
7. Keyboard & Mouse – For navigating and configuring Service Now.

5.Implementation

Task 1: Create UI Policy on Incident

Activity 1: Create UI Policy

1. Log in to the **Service Now instance** with administrative or configuration access.
2. Navigate to **:System UI → UI Policies**
3. Click **New** to create a new UI Policy record.

4. Fill in the following details:

- **Name:** High Impact Control
This name clearly represents the purpose of the policy, which is to apply additional controls when an Incident has a high impact.
- **Table:** Incident
Specifies that the UI Policy applies to Incident records.
- **Active:** true
Enables the UI Policy so it is enforced on the Incident form.

5. Configure the **Conditions** section as follows:

- **Field:** Impact
- **Operator:** is
- **Value:** 1 – High

6. This condition ensures the UI Policy is triggered only when the Incident impact is set to High.

7. And UI policy action Assignment group

8. Check mandatory

9. Set **Reverse if false** to **true**.

This ensures that when the condition is not met, any field changes applied by the policy are automatically reverted.

10. Review the configuration to ensure accuracy.

11. Click **Submit** to save the UI Policy.

servicenow							
All Favorites History Workspaces UI Policies							
Search							
UI Policies Order Search							
All							
	Short description	Table	Conditions	Reverse if false	On load	Updated	Order
☐	UI Policy to hide related feature flags when hidePagination set to true	UX List [sys_ux_list]	hide_pagination=true^EQ	true	true	2021-08-16 14:02:50	0
	Mark fields as mandatory without DB override	Security tasks [sn_vsc_security_task]		false	true	2025-01-28 14:28:42	0
	Always Hide Read-only Related List Field	Relationship [sys_relationship]	sys_idISNOTEMPTY^EQ	false	true	2015-08-21 17:28:31	1
	Hide meta fields	label [sc_ic_question]		true	true	2014-01-21 12:43:35	1
	Set Table column ReadOnly	To-dos Widget Mapping [sn_hr_sp_todos_widget_mapping]		true	true	2018-08-02 15:16:10	1
	Clear save signature before action execution value - when precondition is not Signature	Function [sys_sg_button]	preconditions!=signature^EQ	true	true	2019-04-18 07:01:21	1
	Content vs parameters	HTTP Method [sys_rest_message_fn]	http_method=get^ORhttp_method=delete^ORh...	true	true	2016-12-14 16:50:13	1
	Instance Clone onChange Profile - Hide lock profile	System Clone [clone_instance]	profileISEMPTY^EQ	true	true	2019-08-15 15:10:40	1
1 to 20 of 2,964							

servicenow		All	Favorites	History	Admin	UI Policy - High Impact Control	Search												
UI Policy High Impact Control View: Advanced														Update	Delete				
Table		Incident [Incident]												Application		Global			
														Active		☒			
* Short description		High Impact Control																	
Order														100					
When to Apply														Script					
Conditions														Add Filter Condition		Add OR Clause			
Impact														is		1 - High		AND OR	
If selected, the UI Policy applies to all form views; otherwise the UI Policy is view-specific														Apply the UI policy actions when the form is loaded and when the user changes values on the form					
Global														☒		On load		☒	
Reverse the effects of the UI policy actions when the Conditions evaluate to false														Tables that extend the specified Table inherit this UI Policy					
Reverse if false														☒		Inherit		☐	

UI policy actions specify exactly what actions to take on a specified field. The conditions specified in the UI policy determine when these actions are triggered. [More Info](#)

UI policy: **High Impact Control**

Table: **Incident [incident]**

* Field name: **Assignment group**

Application: **Global**

Mandatory: **True**

Visible: **Leave alone**

Read only: **False**

Clear the field value: ☐

Submit

Task 2: Create UI Policy Action – Urgency

Activity 1: Configure the Urgency Field

1. Open the UI Policy Actions

Go to the **High Impact Control** UI Policy you created earlier. Scroll down to the **UI Policy Actions** related list. This is where you define the specific behavior of fields when the policy conditions are met.

2. Create a New Action

Click **New** to create a UI Policy Action. This lets you configure how individual fields behave when the UI Policy is triggered.

3. Fill in the Action Details

- **Field name:** Urgency
Select the **Urgency** field on the Incident form. This is the field you want to control.
- **Read-only:** true

Check this option to make the Urgency field read-only whenever the UI Policy condition is met (Impact = High). Users will be able

to see the field but cannot change its value.

- **Visible:** Leave this field as is (do not change).

By leaving it unchanged, the field will remain visible on the form, but its behavior will be controlled by the Read-only setting.

4. Submit the Action

Click **Submit** to save the UI Policy Action. The action is now linked to the High Impact Control UI Policy and will automatically apply to the Urgency field when an Incident has High Impact.

5. Optional Verification

To test the action:

- Open an Incident record.
- Set the **Impact** field to **High**.
- Check that the **Urgency** field becomes read-only.
- Change Impact to a different value to confirm that the field reverts back if needed.

The screenshot shows the 'UI Policy Action - New Record' form in ServiceNow. The form is titled 'UI Policy Action - New Record' and includes a search bar and a 'Submit' button. The form is divided into two main sections: 'UI policy' and 'Application'. The 'UI policy' section includes a dropdown for 'UI policy' (set to 'High Impact Control'), a dropdown for 'Table' (set to 'Incident [incident]'), and a dropdown for '* Field name' (set to 'Urgency'). The 'Application' section includes a dropdown for 'Application' (set to 'Global'), a dropdown for 'Mandatory' (set to 'Leave alone'), a dropdown for 'Visible' (set to 'Leave alone'), and a dropdown for 'Read only' (set to 'True'). There is also a checkbox for 'Clear the field value' which is currently unchecked. A 'Submit' button is located at the bottom left of the form.

Task 3: Create on Change Client Script

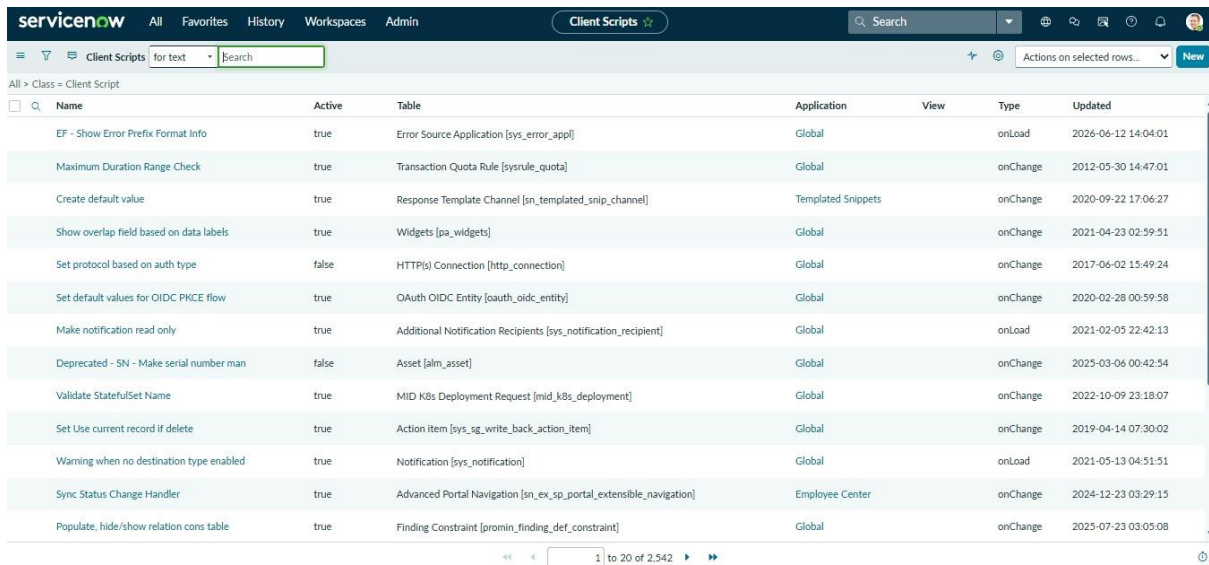
Activity 1: Create Client Script

1. **Navigate to Client Scripts:** Go to System UI → Client Scripts.
2. **Click New** to create a new Client Script.
3. **Fill in the details:**
 - **Name:** Auto set urgency for high impact – clearly shows the purpose.
 - **Table:** Incident – the script runs only on Incident records.
 - **Type:** on Change – executes when a field value changes.
 - **Field name:** Impact – triggers the script when Impact is updated.
 - **Active:** true – ensures the script runs immediately.
4. **Paste the script** in the script editor section.
5. **Click Submit** to save the Client Script.
6. **Verify:** Open an Incident record and change the Impact field to **High**. The Urgency field should update automatically

Script:

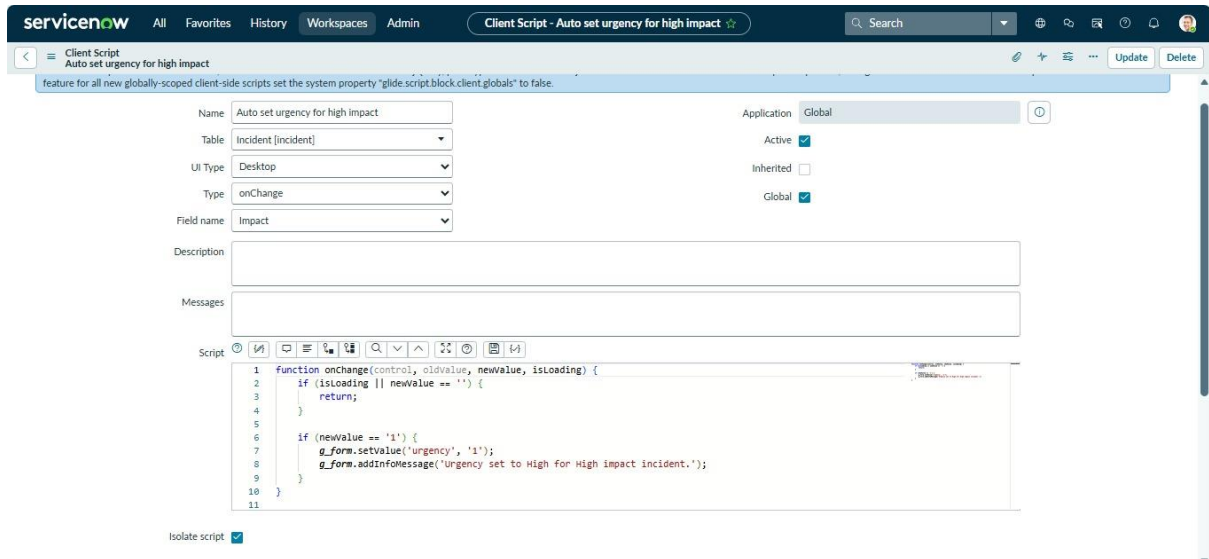
```
function onChange(control, oldValue, newValue, isLoading) {  
    if (isLoading || newValue == '') {  
        return;  
    }  
    if (newValue == '1') {  
        g_form.setValue('urgency', '1');  
        g_form.addInfoMessage('Urgency set to High for High impact incident.');    }  
}
```

5. Click Submit.



The screenshot shows the ServiceNow interface with the 'Client Scripts' list view. The table lists various client scripts with columns for Name, Active, Table, Application, View, Type, and Updated. The 'Name' column is highlighted with a green border.

Name	Active	Table	Application	View	Type	Updated
EF - Show Error Prefix Format Info	true	Error Source Application [sys_error_app]	Global		onLoad	2026-06-12 14:04:01
Maximum Duration Range Check	true	Transaction Quota Rule [sysrule_quota]	Global		onChange	2012-05-30 14:47:01
Create default value	true	Response Template Channel [sn_templated_snip_channel]	Templated Snippets		onChange	2020-09-22 17:06:27
Show overlap field based on data labels	true	Widgets [pa_widgets]	Global		onChange	2021-04-23 02:59:51
Set protocol based on auth type	false	HTTP(s) Connection [http_connection]	Global		onChange	2017-06-02 15:49:24
Set default values for OIDC PKCE flow	true	OAuth OIDC Entity [oauth_oidc_entity]	Global		onChange	2020-02-28 00:59:58
Make notification read only	true	Additional Notification Recipients [sys_notification_recipient]	Global		onLoad	2021-02-05 22:42:13
Deprecated - SN - Make serial number man	false	Asset [alm_asset]	Global		onChange	2025-03-06 00:42:54
Validate StatefulSet Name	true	MID K8s Deployment Request [mid_k8s_deployment]	Global		onChange	2022-10-09 23:18:07
Set Use current record if delete	true	Action item [sys_sg_write_back_action_item]	Global		onChange	2019-04-14 07:30:02
Warning when no destination type enabled	true	Notification [sys_notification]	Global		onLoad	2021-05-13 04:51:51
Sync Status Change Handler	true	Advanced Portal Navigation [sn_ex_sp_portal_extensible_navigation]	Employee Center		onChange	2024-12-23 03:29:15
Populate, hide/show relation cons table	true	Finding Constraint [promin_finding_def_constraint]	Global		onChange	2025-07-23 03:05:08



The screenshot shows the ServiceNow interface for configuring a Client Script. The script is named 'Auto set urgency for high impact' and is associated with the 'Incident' table. The script is active and set to 'onChange' type. The script code is as follows:

```
1 function onChange(control, oldValue, newValue, isLoading) {
2   if (isLoading || newValue == '') {
3     return;
4   }
5
6   if (newValue == '1') {
7     g_form.setValue('urgency', '1');
8     g_form.addInfoMessage('Urgency set to High for High impact incident.');
```

Task 4: Create on Submit Client Script (Save Validation)

Activity 1: Create Client Script

1. **Navigate to Client Scripts:** Go to System UI → Client Scripts in your Service Now instance.
2. **Click New** to create a new Client Script.
3. **Fill in the details:**
 - **Name:** Prevent save if Assigned To missing – clearly indicates that the script prevents saving if the field is empty.
 - **Table:** Incident – applies the script only to Incident records.
 - **Type:** on Submit – executes when the user tries to save the record.
 - **Active:** true – ensures the script is enforced immediately.
4. **Paste the script** into the script editor section. This script should check if the **Assigned To** field is empty and prevent the record from saving if it is.
5. **Click Submit** to save the Client Script.
6. **Verify:** Try saving an Incident without filling the **Assigned To** field. The script should prevent the save and display a warning.

7. Paste the script:

```
function onSubmit() {  
    if (g_form.getValue('impact') == '1' &&  
        g_form.getValue('assigned_to') == "") {  
        g_form.showErrorBox(  
            'assigned_to',  
            'Assigned To is mandatory for High impact incidents.'  
        );  
        return false;  
    }  
}
```

```

return true;

}

```

8. Click **Submit**.

The screenshot shows the ServiceNow interface for configuring a Client Script. The title bar indicates the script is named "Client Script - Prevent save if Assigned To missing". A note at the top states: "feature for all new globally-scoped client-side scripts set the system property 'glide.script.block.client.globals' to false."

The configuration fields are as follows:

- Name:** Prevent save if Assigned To missing
- Table:** Incident [incident]
- UI Type:** Desktop
- Type:** onSubmit
- Application:** Global
- Active:** ☒
- Inherited:** ☐
- Global:** ☒

The **Description** and **Messages** fields are empty.

The **Script** field contains the following JavaScript code:

```

1 function onSubmit() {
2   if (g_form.getValue('impact') == '1' &&
3       g_form.getValue('assigned_to') == '') {
4
5     g_form.showErrorBox(
6       'assigned_to',
7       'Assigned To is mandatory for High impact incidents.'
8     );
9     return false;
10  }
11  return true;
12 }
13

```

The **Isolate script** checkbox is checked.

Task 5: Create on Cell Edit Client Script

Activity 1: Create Client Script

1. **Navigate to Client Scripts:** Go to System UI → Client Scripts in your Service Now instance.
2. **Click New** to create a new Client Script.
3. **Fill in the details:**
 - **Name:** Prevent state change via list edit – clearly indicates that the script prevents users from changing the State field directly in a list.
 - **Table:** Incident – applies the script only to Incident records.
 - **Type:** on Cell Edit – runs whenever a field is edited directly in a list view.
 - **Field name:** State – the script will monitor changes to the State field.
 - **Active:** true – ensures the script is enforced immediately.

4. **Paste the script** into the script editor section. The script should prevent unauthorized changes to the State field when editing from the list view.
5. **Click Submit** to save the Client Script.
6. **Verify:** Open an Incident list and try to change the State field directly in the list. The script should block the change as intended.
7. In the Script field, paste the following code:

```
function onCellEdit(sys IDs, table, old Values, new Value, call back) {  
  
    alert('State cannot be updated using list editing. Please open the Incident.');
```

```
    call back(false);  
  
}
```

8. Click **Submit**.

The screenshot shows the ServiceNow interface for configuring a Client Script. The title bar reads "Client Script - Prevent state change via list edit". A blue informational banner at the top states: "New client-scripts are run in strict mode, with direct DOM access disabled. Access to jQuery, prototype and the window object are likewise disabled. To disable this on a per-script basis, configure this form and add the 'Isolate script' field. To disable this feature for all new globally-scoped client-side scripts set the system property 'glide.script.block.client.globals' to false."

The configuration form includes the following fields:

- Name:** Prevent state change via list edit
- Table:** Incident [Incident]
- UI Type:** Desktop
- Type:** onCellEdit
- Field name:** State
- Application:** Global
- Active:** ☒
- Inherited:** ☐
- Global:** ☒

Below these fields are empty text areas for **Description** and **Messages**.

The **Script** section contains a code editor with the following JavaScript code:

```
1 function onCellEdit(sysIDs, table, oldValues, newValue, callback) {  
2  
3     alert('State cannot be updated using list editing. Please open the Incident.');
```

```
4  
5     callback(false);  
6 }  
7
```

At the bottom, there is a checkbox for **Isolate script** which is currently checked.

Task 6: Testing the Configuration

Activity 1: Test Mandatory Enforcement

1. Navigate to Incident:

Go to Incident → Create New to open a new Incident form.

2. Set the Impact Field:

Select **High** for the **Impact** field. This will trigger the UI Policy and Client Script behaviors you configured earlier.

3. Leave Assigned To Empty:

Do not fill in the **Assigned To** field. This will test the on Submit Client Script that prevents saving if Assigned To is missing.

4. Click Submit:

Attempt to save the Incident record.

5. Verify the Result:

- The Incident should **not be saved** because Assigned To is required when Impact is High.
- An **error message** should appear, indicating that the Assigned To field cannot be empty.
- Confirm that all UI Policy actions, such as read-only fields or automated urgency updates, are working as expected.

The screenshot displays the ServiceNow 'Incident - Create INC0010005' form. The form is titled 'Incident - Create INC0010005' and includes a search bar and navigation tabs. The form fields are organized into two columns. The left column contains fields for Number (INC0010005), Caller (Abel Tuter), Category (Inquiry / Help), Subcategory (-- None --), Service, Service offering, and Configuration item. The right column contains fields for Channel (-- None --), State (New), Impact (1 - High), Urgency (1 - High), Priority (1 - Critical), Assignment group (CAB Approval), and Assigned to. A red error message is displayed below the Assigned to field, stating 'Assigned To is mandatory for High impact incidents.' The form also includes a 'Submit' button and a 'Resolve' button. At the bottom, there is a 'Related Search Results' section with a search bar and a dropdown menu.

Activity 2: Test Successful Save

1. **Fill the Assigned To Field** :Open the Incident form and select a user in the **Assigned To** field. This ensures the on Submit Client Script condition is satisfied.
2. **Click Submit** :Attempt to save the Incident record.
3. **Verify the Result:**
 - The Incident should be **saved successfully** without any errors.
 - Confirm that all other configured behaviors.such as:
 - **Urgency auto-setting** (from the on Change Client Script)
 - **Read-only fields** (from the UI Policy Action)
 - **State restrictions** (from the on CellEdit Client Script) are functioning as expected

The screenshot displays the ServiceNow Incident form for incident INC0010005. The form is organized into two main columns. The left column contains fields for Number (INC0010005), Caller (Abel Tuter), Category (Inquiry / Help), Subcategory (None), Service, Service offering, Configuration item, Short description (UI Policy), and Description. The right column contains fields for Channel (None), State (In Progress), Impact (1 - High), Urgency (1 - High), Priority (1 - Critical), Assignment group (CAB Approval), and Assigned to (Bernard Laboy). Below the form fields, there is a 'Related Search Results' section and a 'Notes' section with tabs for 'Related Records' and 'Resolution Information'. The 'Work notes' section is also visible at the bottom.

Activity 3: Reverse Condition Test

1. **Open an Existing Incident:** Go to Incident → Open and select an Incident where **Impact** is currently set to **High**.
2. **Change Impact:** Update the **Impact** field from **High** to **Medium**. This will test the **Reverse if false** functionality of your UI Policy and the behavior of your Client Scripts.
3. **Verify the Field Behavior:**
 - **Assigned To** should no longer be mandatory.
 - **Urgency** should now be editable, meaning the read-only UI Policy action is reversed.
4. **Save the Record:**

Click **Submit** to save the changes. The record should save successfully without errors, confirming that the reverse condition works as intended.

The screenshot displays the ServiceNow Incident form for record INC0010007. The form is organized into two main columns. The left column contains fields for Number (INC0010007), Caller (Abraham Lincoln), Category (Inquiry / Help), Subcategory (-- None --), Service, Service offering, Configuration item, Short description (Policy), and Description. The right column contains dropdown menus for Channel (-- None --), State (New), Impact (2 - Medium), Urgency (1 - High), Priority (2 - High), Assignment group, and Assigned to. Below the form fields, there is a 'Related Search Results' section. At the bottom, there are tabs for 'Notes', 'Related Records', and 'Resolution Information'. The 'Notes' tab is currently selected, showing a 'Watch list' and a 'Work notes list' section. An 'Add me' button is visible in the bottom right corner of the form area.

Activity 4: Test List Edit Blocking

1. **Navigate to the Incident List:** Go to Incident → All to view all Incident records in a list view.
2. **Attempt a List Edit:**
 - Locate any Incident in the list.
 - **Double-click the State field** to try editing it directly from the list.
3. **Verify the Behavior:**
 - An **alert message** should appear, indicating that direct changes to the State field are not allowed.
 - The **State value should remain unchanged**, confirming that the onCellEdit Client Script is working correctly.

servicenow Incidents											
All											
	Number	Opened	Short description	Caller	Priority	State	Category	Assignment group	Assigned to	Updated	Updated by
	INC0010007	2026-09-21 08:32:35	Policy	Abraham Lincoln	2 - High	New	Inquiry / Help	(empty)	(empty)	2026-09-21 08:33:34	admin
	INC0010006	2026-09-21 08:31:29	UI	Adela Cervantsz	2 - High	New	Inquiry / Help	(empty)	(empty)	2026-09-21 08:32:33	admin
	INC0010005	2026-09-21 08:22:24	UI Policy	Abel Tuter	1 - Critical	In Progress	Inquiry / Help	CAB Approval	Bernard Laboy	2026-09-21 08:28:15	admin
	INC0009009	2018-08-30 01:06:16	Unable to access the shared folder.	David Miller	4 - Low	New		(empty)	(empty)	2018-12-12 23:30:24	admin
	INC0009005	2018-08-31 21:35:21	Email server is down.	David Miller	1 - Critical	New	Software	(empty)	(empty)	2018-12-12 23:18:55	admin
	INC0009004	2018-09-01 06:13:30	Defect tracking tool is down.	David Miller	3 - Moderate	Closed	Software	(empty)	(empty)	2026-06-12 15:58:47	system
	INC0009003	2018-08-30 02:17:32	Cannot sign into the company portal app	David Miller	3 - Moderate	Closed	Inquiry / Help	(empty)	(empty)	2018-12-12 23:39:53	admin
	INC0009002	2018-09-16 05:49:23	My computer is not detecting the headphone device	David Miller	3 - Moderate	Closed	Hardware	(empty)	(empty)	2026-06-12 15:58:47	system

dev302098.service-now.com says
State cannot be updated using list editing. Please open the Incident.

OK

Actions on selected rows... New

Number	Opened	Short description	Assignment group	Assigned to	Updated	Updated by				
INC0010007	2026-09-21 08:32:35	Policy	Abraham Lincoln	2 - High	New	Inquiry / Help	(empty)	(empty)	2026-09-21 08:33:34	admin
INC0010006	2026-09-21 08:31:29	UI	Adela Cervantsz	2 - High	New	Inquiry / Help	(empty)	(empty)	2026-09-21 08:32:33	admin
INC0010005	2026-09-21 08:22:24	UI Policy	Abel Tuter	1 - Critical	In Progress	Inquiry / Help	CAB Approval	Bernard Laboy	2026-09-21 08:28:15	admin
INC0009009	2018-08-30 01:06:16	Unable to access the shared folder.	David Miller	4 - Low	In Progress	Inquiry / Help	(empty)	(empty)	2018-12-12 23:30:24	admin
INC0009005	2018-08-31 21:35:21	Email server is down.	David Miller	1 - Critical	New	Software	(empty)	(empty)	2018-12-12 23:18:55	admin
INC0009004	2018-09-01 06:13:30	Defect tracking tool is down.	David Miller	3 - Moderate	Closed	Software	(empty)	(empty)	2026-06-12 15:58:47	system
INC0009003	2018-08-30 02:17:32	Cannot sign into the company portal app	David Miller	3 - Moderate	Closed	Inquiry / Help	(empty)	(empty)	2018-12-12 23:39:53	admin
INC0009002	2018-09-16 05:49:23	My computer is not detecting the headphone device	David Miller	3 - Moderate	Closed	Hardware	(empty)	(empty)	2026-06-12 15:58:47	system

1 to 20 of 70

Activity 5: Test Form-Based Update

- Open the Incident Record:** Navigate to the same Incident record you used in previous tests.
- Change the State:** On the **Incident form**, update the **State** field to a new value. This tests that changes via the form (not the list) are allowed.
- Click Update:** Save the record by clicking **Update**.
- Verify the Result:**
 - The **State change should be saved successfully**.
 - Confirm that other fields, such as **Assigned To** and **Urgency**, still follow the configured UI Policy and Client Script rules.

Incident - INC0009009

Number: INC0009009

Caller: David Miller

Category: Inquiry / Help

Subcategory: -- None --

Service: -- None --

Description: Unable to access the shared folder. Please provide access.

Channel: -- None --

State: In Progress

Impact: 3 - Low

Urgency: 2 - Medium

Priority: 4 - Low

Assignment group: -- None --

Assigned to: -- None --

Related Search Results

Work notes

Comments (Customer visible) Post

Activities: 2 System Administrator

Work notes • 2018-12-12 23:30:24

Number	Opened	Short description	Caller	Priority	State	Category	Assignment group	Assigned to	Updated	Updated by
INC0010007	2026-09-21 08:32:35	Policy	Abraham Lincoln	2 - High	New	Inquiry / Help	(empty)	(empty)	2026-09-21 08:33:34	admin
INC0010006	2026-09-21 08:31:29	UI	Adela Cervantsz	2 - High	New	Inquiry / Help	(empty)	(empty)	2026-09-21 08:32:33	admin
INC0010005	2026-09-21 08:22:24	UI Policy	Abel Tuter	1 - Critical	In Progress	Inquiry / Help	CAB Approval	Bernard Laboy	2026-09-21 08:28:15	admin
INC0009009	2018-08-30 01:06:16	Unable to access the shared folder.	David Miller	4 - Low	In Progress	Inquiry / Help	(empty)	(empty)	2026-09-21 08:42:07	admin
INC0009005	2018-08-31 21:35:21	Email server is down.	David Miller	1 - Critical	New	Software	(empty)	(empty)	2018-12-12 23:18:55	admin
INC0009004	2018-09-01 06:13:30	Defect tracking tool is down.	David Miller	3 - Moderate	Closed	Software	(empty)	(empty)	2026-06-12 15:58:47	system
INC0009003	2018-08-30 02:17:32	Cannot sign into the company portal app	David Miller	3 - Moderate	Closed	Inquiry / Help	(empty)	(empty)	2018-12-12 23:39:53	admin
INC0009002	2018-09-16 05:49:23	My computer is not detecting the headphone device	David Miller	3 - Moderate	Closed	Hardware	(empty)	(empty)	2026-06-12 15:58:47	system
INC0009001	2018-09-11 20:56:26	Unable to post content on a Wiki page	David Miller	3 - Moderate	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:32:42	admin
INC0008112	2019-07-29 11:48:43	Assessment : ATF Assessor	survey user	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2019-07-29 11:49:28	admin

1 to 20 of 70

7 .Conclusion

The Implement Client Script & UI Policy (Incident) micro project effectively demonstrates how **UI Policies** and **Client Scripts** can work together to enforce dynamic field behavior, automate updates, and prevent incorrect data submission on Incident forms. By using on Change, on Submit, and on Cell Edit scripts alongside UI Policy actions, the project ensures fields like **Assigned To** and **Urgency** respond appropriately to changes in **Impact**, maintaining

clean and consistent data. This solution not only enforces mandatory fields and controls user interactions but also remains lightweight, efficient, and easy to implement, making it ideal for short-duration micro projects while showcasing practical best practices for improving form usability and data integrity in Service Now.